

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Validador de Prompts y Tokens

PATRÓN ARQUITECTÓNICO: Single-File Dual-Layer (2 archivos máximo)

Dada la restricción estricta de condensación, se adopta una arquitectura minimalista de dos capas desacopladas:

- Capa de Presentación: index.html (SPA autocontenida)
- Capa de Servicio: main.py (API REST con FastAPI)

FRONTEND (index.html):

Arquitectura SPA monolítica autocontenida. Se inyecta Tailwind CSS vía CDN para estilos utilitarios sin build step. La lógica de estado de la UI (loading, error, results) se gestiona con JavaScript vanilla usando el patrón de máquina de estados simple. La comunicación con el backend usa fetch nativa con async/await. Se implementa validación client-side antes de cualquier llamada de red (fail-fast pattern). El diseño visual sigue principios de UI minimalista: área de texto prominente, feedback visual inmediato con clases Tailwind condicionales (borde rojo en error, botón deshabilitado en carga), y panel de resultados con 3 tarjetas métricas.

BACKEND (main.py):

Microservicio FastAPI de endpoint único. Configuración CORS permisiva para orígenes localhost (puertos 3000, 5500, 8080, file://) permitiendo consumo directo desde el navegador sin servidor de archivos. El endpoint POST / api/analyze implementa lógica de negocio pura: cálculo de tokens = $\text{len}(\text{words}) * 1.3$ redondeado a entero, latencia simulada con `asyncio.sleep(random.uniform(0.5, 2.0))` no bloqueante. Modelo Pydantic para validación del payload de entrada. Lógica de estado: tokens ≤ 1000 = Óptimo, tokens > 1000 = Alerta.

SEGURIDAD:

- Validación de payload con Pydantic (type safety, rechazo de inputs malformados)
- CORS restringido a localhost únicamente (no wildcard en producción)
- Sin persistencia de datos (stateless, sin superficie de ataque de BD)
- Sanitización implícita al operar solo sobre conteo de palabras
- Rate limiting no requerido en scope (herramienta interna)

TECNOLOGÍAS:

- Backend: Python 3.10+, FastAPI, Uvicorn, Pydantic v2
- Frontend: HTML5, Tailwind CSS (CDN), JavaScript ES2020 (fetch, async/await)
- Comunicación: HTTP/JSON REST
- Sin base de datos, sin autenticación, sin dependencias externas adicionales

DECISIONES DE DISEÑO CLAVE:

1. `asyncio.sleep()` garantiza no bloqueo del event loop de FastAPI durante la simulación de latencia
2. El cálculo $\text{tokens} = \text{palabras} * 1.3$ es determinista y reproducible
3. El panel de resultados se renderiza dinámicamente en el DOM sin frameworks reactivos
4. Un único archivo Python elimina la necesidad de módulos separados de config/routes

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: WARNING (Forzado) | Iteraciones: 3

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 26,659

Tokens de salida (output): 9,380

Total tokens: 36,039

Horas-hombre estimadas: ~3.3 horas

Modelo utilizado: claude-sonnet-4-6